

Examen Final

Paradigmas de Lenguajes de Programación
Departamento de Computación, FCEyN, UBA
25 de julio de 2025

1a	1b	1c	2	3	4a	4b	5a	5b	Nota
----	----	----	---	---	----	----	----	----	------

Incluir nombre, fecha y firma en todas las hojas.

Ejercicio 1 (Programación funcional)

Considerar la siguiente función:

```
foldu z f []      = z
foldu z f (x : xs) = f x (foldu z f xs)
```

- Dar el tipo de `foldu`.
- Definir `foldr` sin usar recursión explícita, usando `foldu`.
- Definir `foldu` sin usar recursión explícita, usando `foldr`.

Ejercicio 2 (Razonamiento ecuacional)

Notamos `Nat` al tipo de los naturales (enteros no negativos). Contamos con la operación que devuelve el máximo entre dos naturales:

```
max :: Nat -> Nat -> Nat
max x y = if x > y then x else y
```

Se asume ya demostrado que la operación `max` es conmutativa, asociativa y que 0 es el elemento neutro, es decir:

- Conmutatividad:** $\forall x y :: \text{Nat}. \text{max } x y = \text{max } y x$.
- Asociatividad:** $\forall x y z :: \text{Nat}. \text{max } x (\text{max } y z) = \text{max } (\text{max } x y) z$.
- Elemento neutro:** $\forall x :: \text{Nat}. \text{max } 0 x = x$.

Se definen además dos funciones `maxr` y `maxl` para calcular el máximo elemento de una lista de números naturales:

```
maxr :: [Nat] -> Nat
{MRO} maxr []      = 0
{MR1} maxr (x : xs) = max x
                    (maxr xs)

maxl :: Nat -> [Nat] -> Nat
{MLO} maxl ac []      = ac
{ML1} maxl ac (x : xs) =
    maxl (max x ac) xs
```

Las funciones `(++)` y `reverse` son las usuales:

```
(++) :: [a] -> [a] -> [a]
{A0} []      ++ ys = ys
{A1} (x : xs) ++ ys = x : (xs++ys)

reverse :: [a] -> [a]
{R0} reverse []      = []
{R1} reverse (x : xs) = reverse xs ++ [x]
```

Demostrar que $\forall xs :: [\text{Nat}]. \text{maxr } (\text{reverse } xs) = \text{maxl } 0 \text{ xs}$.

Ejercicio 3 (Programación lógica)

Un *árbol general* es o bien el símbolo x o bien una lista de árboles generales. Por ejemplo, los siguientes son árboles generales:

x $[]$ $[x, x]$ $[[x, x], x, [x, x]]$ $[x, [x, x, [x, [], [x, x]], x], [x, []]]$

Definir en Prolog un predicado `arbolGeneral(-A)` que genere todos los posibles árboles generales.

Ejercicio 4 (Deducción natural)

Se define un nuevo conectivo binario $\oplus$, con las siguientes reglas de introducción y eliminación:

$$\frac{\Gamma, \neg\tau, \neg\sigma \vdash \perp}{\Gamma \vdash \tau \oplus \sigma} \oplus_i \qquad \frac{\Gamma \vdash \tau \oplus \sigma \quad \Gamma, \tau \vdash \perp \quad \Gamma, \sigma \vdash \perp}{\Gamma \vdash \perp} \oplus_e$$

Dar derivaciones para los siguientes secuentes:

- a) $\tau \vee \sigma \vdash \tau \oplus \sigma$
- b) $\tau \oplus \sigma \vdash \tau \vee \sigma$

Ejercicio 5 (Resolución)

Dar ejemplos de fórmulas que producen el siguiente comportamiento en el método de resolución.

- a) Hay secuencias de aplicaciones de la regla de resolución que producen infinitas cláusulas distintas. **Hay** una refutación que conduce a la cláusula vacía.
- b) Hay secuencias de aplicaciones de la regla de resolución que producen infinitas cláusulas distintas. **No hay** una refutación que conduce a la cláusula vacía.